



第5章 进程同步机制

- 本章内容
 - 1. 进程互斥、同步的基本概念
 - 2. 实现进程互斥的软硬件方法
 - 3. 锁和条件变量的作用和实现
 - 4. 信号量机制解决互斥同步问题
 - 5. 经典同步问题
 - 6. 各种进程通信机制的原理和区别





进程同步

- 进程同步协调进程间的相互制约关系，使它们按照预期的方式执行的过程。
- 前提
 - 进程是并发执行的，进程间存在着相互制约关系
 - 并发的进程对系统共享资源进行竞争
 - 进程通信，过程中相互发送的信号称为消息或事件
- 两种相互制约形式
 - 间接相互制约关系（互斥）：进程排他性地访问共享资源
 - 直接相互制约关系（同步）：进程间的合作，比如管道通信





一个并发问题

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include "common.h"
4
5  volatile int counter = 0;
6  int loops;
7
8  void *worker(void *arg) {
9      int i;
10     for (i = 0; i < loops; i++) {
11         counter++;
12     }
13     return NULL;
14 }
15
```



一个并发问题

```
16  int
17  main(int argc, char *argv[])
18  {
19      if (argc != 2) {
20          fprintf(stderr, "usage: threads <value>\n");
21          exit(1);
22      }
23      loops = atoi(argv[1]);
24      pthread_t p1, p2;
25      printf("Initial value : %d\n", counter);
26
27      Pthread_create(&p1, NULL, worker, NULL);
28      Pthread_create(&p2, NULL, worker, NULL);
29      Pthread_join(p1, NULL);
30      Pthread_join(p2, NULL);
31      printf("Final value      : %d\n", counter);
32      return 0;
33  }
```





一个并发问题

```
$ gcc -o thread thread.c -Wall -pthread
$ ./thread 1000
Initial value: 0
Final value : 2000

$ ./thread 100000
Initial value: 0
Final value : 143012
$ ./thread 100000
Initial value: 0
Final value : 137128
$ ./thread 100000
Initial value: 0
Final value : 200000
```

- 当loops输入为N时，预计程序输出为2N
- 程序运行不但会产生错误，而且结果不同
- 为什么会出现这种状况？
 - 增加共享计数器的代码由3条指令构成
 - 假设counter存放于地址0x8049a1c

```
100 mov 0x8049a1c, %eax
105 add $0x1, %eax
108 mov %eax, 0x8049a1c
```





核心问题：不可控的调度

- 假设mov占5字节， add占3字节

OS	线程 1	线程 2	指令执行后		
			PC	%eax	counter
	在临界区之前 mov 0x8049a1c, %eax add \$0x1, %eax		100	0	50
			105	50	50
			108	51	50
中断 保存 T1 的状态 恢复 T2 的状态			100	0	50
		mov 0x8049a1c, %eax add \$0x1, %eax mov %eax, 0x8049a1c	105	50	50
			108	51	50
			113	51	51
中断 保存 T2 的状态 恢复 T1 的状态			108	51	51
	mov %eax, 0x8049a1c		113	51	51



临界区问题

- **临界区**：访问共享资源的一段代码，通常资源是一个变量或数据结构
- 假设系统中有 n 个进程 $\{p_0, p_1, \dots, p_{n-1}\}$
- 每个进程都有临界区代码段
 - 进程可能会更改公共变量、更新表、写入文件等
 - 当一个进程在临界区时，其他任何进程都不可能在其临界区中
- 临界区问题需通过设计协议来解决
 - 每个进程必须在进入区中提出进入临界区的申请，在临界区之后有退出区，其余的是剩余区域

do {

entry section

critical section

exit section

remainder section

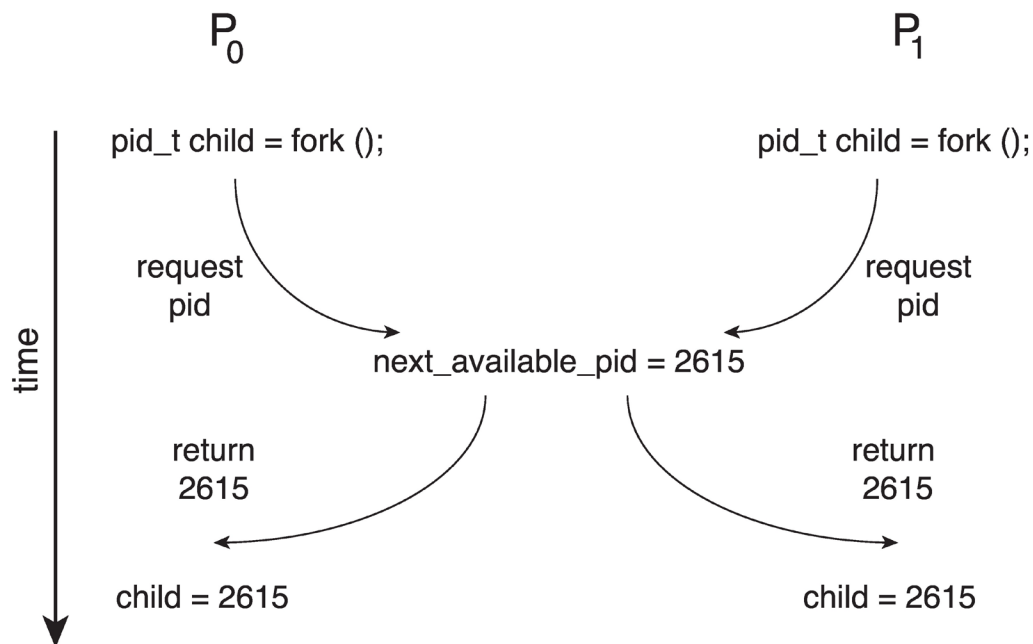
} while (true);





竞态条件(Race Condition)

- P_0 和 P_1 进程使用`fork()`系统调用创建子进程
- `next_available_pid`存在竞态条件



- 除非有机制阻止 P_0 和 P_1 访问变量`next_available_pid`, 否则可能将相同的pid分配给两个不同的进程
- **竞态条件:** 多个进程/线程同时进入临界区, 执行结果与访问发生的特定顺序有关



解决临界区问题

- 解决临界区问题要满足的条件：
 - **互斥** — 如果进程 P_i 正在其临界区，则其他进程不能同时在其临界区
 - **前进（进展性）** — 如果没有进程正在其临界区，且存在希望进入临界区的进程，则必须允许其中一个进程进入临界区，保证程序执行的进展
 - **有限等待**（有界等待） — 一个进程发出进入临界区的请求后，其他进程被允许进入其临界区的次数必须是有限的。
- 假设每个进程以非零速度执行。
- 对 n 个进程的相对速度没有任何限制。



临界区访问规则

entry section
critical section
exit section
remainder section

- 1.空闲则入**：没有线程在临界区时，任何线程**可**进入
- 2.忙则等待**：有线程在临界区时，其他线程均**不**能进入临界区
- 3.有限等待**：等待进入临界区的线程**不能**无限期等待
- 4.让权等待（可选）**：不能进入临界区的线程，应释放**CPU**（如转换到阻塞状态）





软件方案1

- 假设有两个进程
- 假设load和store机器语言指令是原子的，即不能被中断
- 这两个进程共享一个变量：
 - `int turn;`
- 变量turn表示轮到谁进入临界区
- 初始时，`turn = 0`

P0进程

```
while(turn != 0);  
critical section;  
turn = 1;  
remainder section;
```

P1进程

```
while(turn != 1); // 进入区  
critical section; // 临界区  
turn = 0;         // 退出区  
remainder section; // 剩余区
```

公共变量turn,
用于指示被允许
进入的进程编号



软件方案的正确性

■ 满足互斥

进程 P_0 进入临界区，当且仅当：

$$\text{turn} = 0$$

且 turn 不能同时为0和1

■ 进程的“进展性”要求是否满足？

- $\text{turn} = 1$ ，如果进程 P_0 想进入临界区而进程 P_1 不想进入，进程 P_0 能进入吗？

■ “有界等待” 要求是否能满足呢？



双标志法

共享变量

bool flag[2]

flag[0] = flag[1] = 0

flag[i] = 1 //表示进程Pi想进入临界区

P0进程

P1进程

<pre>bool flag[2]; while(flag[1]); flag[0] = true; critical section; flag[0] = false; remainder section;</pre>	<pre>while(flag[0]); // 进入区 flag[1] = true; critical section; // 临界区 flag[1] = false; // 退出区 remainder section; // 剩余区</pre>
---	---

先检查，违背“忙则等待”（或互斥）



双标志法

P0进程

P1进程

<pre>bool flag[2]; flag[0] = true; while(flag[1]); critical section; flag[0] = false; remainder section;</pre>	<pre>flag[1] = true; while(flag[0]); // 进入区 critical section; // 临界区 flag[1] = false; // 退出区 remainder section; // 剩余区</pre>
---	---

布尔型数组flag[],
flag[i]为true, 进程已进入临界区
flag[i]为false, 进程未进入临界区

后检查, 违背“有限等待”





软件方案 — Peterson算法

P0进程

P1进程

<pre>bool flag[2]; int turn = 0; flag[0] = true; turn = 1; while(flag[1] && turn==1); critical section; flag[0] = false; remainder section;</pre>	<pre>flag[1] = true; turn = 0; while(flag[0] && turn==0); // 进入区 critical section; // 临界区 flag[1] = false; // 退出区 remainder section; // 剩余区</pre>
--	--

布尔型数组flag[],
flag[i]为true, 进程已进入临界区
flag[i]为false, 进程未进入临界区
公共变量turn, 被允许进入的进程编号



软件方案 — Peterson算法

- 进程 P_0 的算法

```
while (true) {
```

```
    flag[0] = true;
```

```
    turn = 1;
```

```
    while (flag[1] && turn  
           == 1);
```

```
    /* critical section */
```

```
    flag[0] = false;
```

```
    /* remainder section */
```

```
}
```

- 两个进程共享两个变量：
int turn;
boolean flag[2];
- 变量turn表示轮到谁进入临界区
- flag数组用于指示进程是否准备好进入临界区
- flag[0] = true表示进程 P_0 准备好进入临界区



Peterson算法

```
enum boolean {false, true};
boolean flag[2] = {false, false}; int turn;

P0:  do {
        flag[0] = true; turn = 1;
        while (flag[1] && turn == 1) ;
        P0的临界区代码CS0;
        flag[0] = false;
        P0的其他代码;
    } while (true);

P1:  do {
        flag[1] = true; turn = 0;
        while (flag[0] && turn == 0) ;
        P1的临界区代码CS1;
        flag[1] = false;
        P1的其他代码;
    } while (true);
```

可保证安全
但进程必须限定
为2个，超过2
个，turn就无法
正确设置，等待
的条件也无法表
达





Peterson算法的正确性

- 可以证明三个临界区要求都得到了满足：

- 1. 互斥性得以保留

当且仅当以下条件满足时，进程 P_0 才能进入临界区：
要么 $\text{flag}[1] = \text{false}$ ，要么 $\text{turn} = 0$

- 2. 满足进展要求
- 3. 满足有界等待要求



Peterson方案

- 只能为两个进程提供解决方案
- 怎样修改算法能使其支持10个进程？
 - Eisenberg和McGuire算法
 - Bakery算法
- 该方案需要“忙等”
 - 进程不断浪费CPU周期来询问是否能够进入临界区





Peterson方案和现代架构

- 作为一个算法，Peterson算法是有作用的，但无法保证在现代架构上正常运行
 - 为了提高性能，处理器和/或编译器可能会重新排序没有依赖关系的操作
- 理解它为什么不起作用有助于更好地理解竞态条件
- 对于单线程来说，算法是正确的，因为结果总是相同的
- 对于多线程来说，重新排序可能会产生不一致或意外的结果！





现代架构下的代码示例

两个线程共享以下数据

```
boolean flag = false;
```

```
int x = 0;
```

线程1:

```
while (!flag);
```

```
print x;
```

线程2:

```
x = 100;
```

```
flag = true;
```

■ 期望的输出是什么？ 100



现代架构下的代码示例

- flag和x是两个彼此独立的变量
- 线程2的指令可能被排序为：

flag = true;

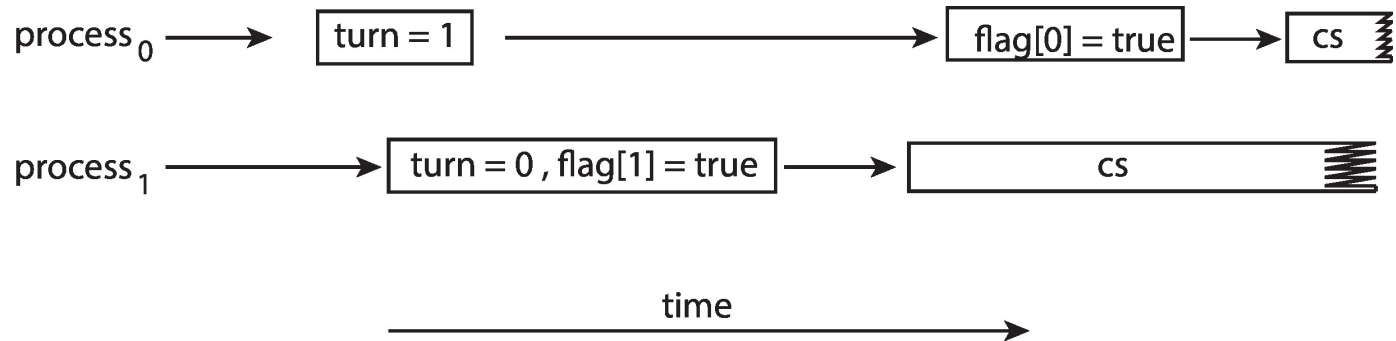
x = 100;

- 实际的输出可能是： 0



修订的Peterson方案

■ Peterson方案中，指令重排的效果



- 使得两个进程可以同时进入它们的临界区！
- 为了确保Peterson方案在现代计算机架构上能够正常工作，必须使用**内存屏障**





同步的硬件支持

- 许多系统提供硬件指令来支持临界区问题
- 单处理器 – 采用禁用中断的方式
 - 当前运行的代码将无法被抢占
 - 可行吗？
 - 在多处理器系统上禁中断通常效率太低
 - 使用此方法的操作系统不具备广泛的可扩展性
- 三种形式的硬件支持：
 - 内存屏障
 - 硬件指令
 - 原子变量





基于中断解决临界区问题

```
do {  
    ...  
    entry section      ← 关中断;  
    critical section   ← 临界区;  
    exit section       ← 开中断;  
    remainder section  ...  
} while (true);
```

- 临界区的问题可以得到解决吗？
 - 如果临界区是运行一个小时的代码，那该怎么办？
 - 某些进程是否可能饿死 — 从未进入它们的临界区？
 - 如果有两个CPU会怎样？



内存屏障指令

- **内存屏障指令**用于确保在执行任何后续的加载或存储操作之前，前面所有的加载和存储指令都已完成。
- 因此，即使指令被重新排序，内存屏障也确保存储操作在内存中完成并对其他处理器可见，然后才执行后来的加载或存储操作。





内存屏障示例

两个线程共享以下数据

boolean flag = false;

int x = 0;

线程1:

while (!flag)

memory_barrier();

print x;

线程2:

x = 100;

memory_barrier();

flag = true;

- 确保flag的值在x之前加载
- 确保x的赋值在flag的赋值之前完成





内存屏障

■ 指令类型

- Load Barrier: 读屏障, 确保屏障前读完
- Store Barrier: 写屏障, 确保屏障前写完
- Full Barrier: 全屏障, 同时包含读写屏障

■ 不同语言和系统中的内存屏障

- 编写内核程序和应用程序都可能用到
- Linux内核
 - `mb()`, `rmb()`, `wmb()`等函数





内存屏障

■ 不同语言和系统中的内存屏障

■ C++ 11

- `std::atomic_thread_fence()`
- `std::memory_order_xxxx`

■ GCC

- `__sync_synchronize()`

■ MSVC

- `_mm_mfence()`
- `_mm_lfence()`
- `_mm_sfence()`





硬件指令

- 特殊的硬件指令允许我们**原子地**测试并修改单词的内容，或者原子地交换两个单词的内容（不被中断）。
 - test-and-set指令
 - compare-and-swap比较并交换指令





test-and-set指令

■ 定义:

```
boolean test_and_set(boolean *lock)
{
    boolean old = *lock;
    *lock = true;
    return old;
}
```

■ 特点:

- 以原子方式执行
- 返回参数的原始值
- 将参数的值设置为true





test-and-set指令解决临界区问题

- 初始时，共享变量lock = false;

```
do {  
    while (test_and_set(&lock)) ;  
    /* do nothing */  
  
    /* critical section */  
  
    lock = false;  
    /* remainder section */  
} while (true);
```

- 采用忙等
- 是否可以解决临界区问题？





compare_and_swap 指令

- 定义:

```
int compare_and_swap(int *value, int expected,
                    int new_value)
{
    int temp = *value;
    if (*value == expected)
        *value = new_value;
    return temp;
}
```

- 特点:
- 以原子方式执行
- 返回参数值的原始值
- 仅在 `*value == expected` 为真时，将变量值设置为参数 `new_value` 的值。即交换仅在此条件下进行。





compare_and_swap解决临界区问题

- 初始时，共享变量lock = 0;

```
while (true){  
    while (compare_and_swap(&lock, 0, 1) != 0) ;  
    /* do nothing */  
  
    /* critical section */  
  
    lock = 0;  
  
    /* remainder section */  
}
```

- 采用忙等
- 是否可以解决临界区问题？





原子变量

- 原子变量：是一种特殊的变量，它的任何单个操作(如读取、写入等)都是原子的，这些操作在执行过程中不会被其它进程打断。
- 例如：
 - 假设 `sequence` 是一个原子变量
 - 假设 `increment()` 是对原子变量 `sequence` 的操作
- 指令：`increment(&sequence);`
可以确保 `sequence` 在不被中断的情况下递增



原子变量的实现

■ 函数increment()的实现:

```
void increment(atomic_int *v)
{
    int temp;
    do {
        temp = *v;
    } while (temp != (compare_and_swap(v,temp,temp+1)));
}
```





原子变量的使用

- 在C++11中,可以使用std::atomic模板来定义原子变量,保证对该变量的访问是原子的。

```
#include <atomic>
```

```
std::atomic<int> counter(0); // 定义一个原子整型变量
```

```
void increment() {  
    counter.fetch_add(1); // 原子地递增变量值  
}
```

```
void decrement() {  
    counter.fetch_sub(1); // 原子地递减变量值  
}
```

- **fetch_add**和**fetch_sub**这两个原子操作对整数counter进行原子增减
- std::atomic还提供了load、store、exchange等原子操作,可以原子地读写变量





锁机制

- 硬件指令解决临界区问题逻辑复杂，且不易移植
- 操作系统设计者构建了软件工具来解决临界区问题
- 最简单的是互斥锁
 - 布尔变量表示锁是否可用
 - 互斥锁是在硬件指令上进行的封装，提供了更高级的互斥语义
- 互斥锁提供两个操作：
 - 获取锁 `acquire()`
 - 释放锁 `release()`
- `acquire()`和`release()`是原子执行的
 - 通常通过硬件原子指令（例如`compare-and-swap`）实现





互斥锁解决临界区问题

■ 进程 P_i 的代码

```
while (true) {  
    acquire lock  
    critical section  
    release lock  
    remainder section  
}
```



CAS实现有限等待互斥锁

```
while (true) {
    waiting[i] = true;
    key = 1;
    while (waiting[i] && key == 1)
        key = compare_and_swap(&lock, 0, 1);
    waiting[i] = false;
    /* critical section */
    j = (i + 1) % n;
    while ((j != i) && !waiting[j])
        j = (j + 1) % n;
    if (j == i)
        lock = 0;
    else
        waiting[j] = false;
    /* remainder section */
}
```





Pthread锁

- POSIX库将锁称为互斥量(mutex)，它被用来提供线程之间的互斥

```
1 pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;  
2  
3 Pthread_mutex_lock(&lock); // wrapper for pthread_mutex_lock()  
4 balance = balance + 1;  
5 Pthread_mutex_unlock(&lock);
```

- POSIX的lock和unlock函数会传入一个变量，因此可以用不同的锁来保护不同的变量





怎样实现一个锁？

- 如何评价锁的效果？
 - 提供互斥 —— 基本任务
 - 公平 —— 是否有公平机会抢到锁，没有饥饿
 - 性能 —— 时间开销
- 开锁和关锁 **原语**

```
void lock(lock_t *mutex){  
    while(mutex->flag == 1)  
        ; // spin-wait(do nothing)  
    mutex->flag = 1; // set the flag  
}
```

```
void unlock(lock_t *mutex){  
    mutex->flag = 0;  
}
```



利用TAS指令实现自旋锁

```
typedef struct    lock_t {  
    int flag;  
} lock_t;  
  
void init(lock_t *lock) {  
    // 0 indicates that lock is available, 1 that it is held  
    lock->flag = 0;  
}  
  
void lock(lock_t *lock) {  
    while (TestAndSet(&lock->flag, 1) == 1)  
        ; // spin-wait (do nothing)  
}  
  
void unlock(lock_t *lock) {  
    lock->flag = 0;  
}
```



TAS实现锁的有效性

■ 互斥性

- 当只有一个线程时，可以获得锁，并进入临界区；执行完成时，离开临界区，并释放锁
- 当有线程持有锁时，新线程调用lock，会一直自旋等待，直到锁被释放，它将获得锁并进入临界区
- 设置和测试合并为一个原子操作后，确保了只有一个线程可以获得锁

■ 公平性

- 在单CPU上，需要抢占式调度器，否则自旋锁无法使用，因为自旋线程永远不会放弃CPU

■ 性能

- 单CPU情况下，性能开销大，因为当一个线程获得锁后，其余线程在竞争锁时，都会放弃CPU之前自旋一个时间片
- 在多CPU上性能不错，因为临界区一般很短，只需自旋很短的时间





利用CAS指令实现自旋锁

■ Compare-and-exchange指令的伪码描述

```
int CompareAndSwap(int *ptr, int expected, int new) {  
    int actual = *ptr;  
    if (actual == expected)  
        *ptr = new;  
    return actual;  
}
```

■ CAS指令实现锁

```
void lock(lock_t *lock) {  
    while (CompareAndSwap(&lock->flag, 0, 1) == 1)  
        ; // spin  
}
```





利用链接的加载和条件式存储指令实现自旋锁

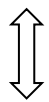
- 在MIPS架构中，可以利用链接式加载(load-linked)和条件式存储(store-conditional)指令，配合使用来支持并发

```
int LoadLinked(int *ptr) {
    return *ptr;
}

int StoreConditional(int *ptr, int value) {
    if (no update to *ptr since LoadLinked to this address) {
        *ptr = value;
        return 1; // success!
    } else {
        return 0; // failed to update
    }
}

void lock(lock_t *lock) {
    while (1) {
        while (LoadLinked(&lock->flag) == 1)
            ; // spin until it's zero
        if (StoreConditional(&lock->flag, 1) == 1)
            return; // if set-it-to-1 was a success: all done
                    // otherwise: try it all over again
    }
}

void lock(lock_t *lock) {
    while (LoadLinked(&lock->flag) || !StoreConditional(&lock->flag, 1))
        ; // spin
}
```





获取并增加指令(fetch-and-add)实现ticket锁

```
int FetchAndAdd(int *ptr) {
    int old = *ptr;
    *ptr = old + 1;
    return old;
}

typedef struct lock_t {
    int ticket;
    int turn;
} lock_t;

void lock_init(lock_t *lock) {
    lock->ticket = 0;
    lock->turn = 0;
}

void lock(lock_t *lock) {
    int myturn = FetchAndAdd(&lock->ticket);
    while (lock->turn != myturn)
        ; // spin
}

void unlock(lock_t *lock) {
    FetchAndAdd(&lock->turn);
}
```

- 该方法可以确保所有线程均可抢到锁，而前面的方法则不能保证





怎样避免过多自旋

- 当一个线程获得锁，则其余线程自旋等待
- 线程自旋检查一个不会改变的值，直到浪费整个时间片
- 解决方案1:
 - 当要自旋时，放弃CPU，进程从运行态变为就绪态
 - 多个线程竞争一把锁时，上下文切换成本依旧很高
 - 可能存在饥饿情况





怎样避免过多自旋

■ 解决方案2: Solaris方案

- 通过维护一个队列来保存等待锁的线程，从而确定谁能抢到锁
- park()让调用线程休眠
- unpark(threadID)唤醒threadID标识的线程

```
typedef struct __lock_t {  
    int flag;  
    int guard;  
    queue_t *q;  
} lock_t;
```

```
void lock_init(lock_t *m) {  
    m->flag = 0;  
    m->guard = 0;  
    queue_init(m->q);  
}
```

```
void lock(lock_t *m) {  
    while (TestAndSet(&m->guard, 1) == 1)  
        ; //acquire guard lock by spinning  
    if (m->flag == 0) {  
        m->flag = 1; // lock is acquired  
        m->guard = 0;  
    } else {  
        queue_add(m->q, gettid());  
        m->guard = 0;  
        park();  
    }  
}
```

```
void unlock(lock_t *m) {  
    while (TestAndSet(&m->guard, 1) == 1)  
        ; //acquire guard lock by spinning  
    if (queue_empty(m->q))  
        m->flag = 0; // let go of lock; no one wants it  
    else  
        unpark(queue_remove(m->q)); // hold lock  
                                        // (for next thread!)  
    m->guard = 0;  
}
```

不能完全避免自旋等待，但等待时间有限



自旋锁分析

■ 临界区

■ 若临界区期间发生切换会怎样？

- 锁未开，CPU切至另一进程，仍被自旋锁挡，转到时间片结束。
- 并不破坏互斥逻辑，但CPU的浪费难以忍受。
- 会出现优先级反转，实时系统不可接受。

■ 在内核态中临界区通常禁止抢占

- 如何实现？

```
do {  
    acquire(lock);  
    临界区  
    release(lock);  
    剩余区  
} while(true);
```





条件变量

- 多线程程序中，如何等待一个条件？
 - 自旋等待，直到条件满足
 - 使用条件变量，来等待一个条件变为真
- 条件变量：
 - 是一个显示队列，当条件不满足时，线程可以将自己加入队列；当有其他线程改变了条件状态时，则可以唤醒等待线程
 - `wait()`操作使调用该操作的线程睡眠。
 - `signal()`操作唤醒条件变量上睡眠的线程。



条件变量

- 条件变量与一个互斥锁（**mutex**）配合使用，互斥锁用于保护共享条件变量所依赖的共享数据。

```
typedef struct {  
    int value; //进程数  
    struct process *wait_queue;  
} cond;
```

```
cond_wait(cond, mutex) {  
    // 将当前线程加入 cond 的等待队列  
    enqueue(current_thread, cond->wait_queue);  
  
    // 原子地：释放 mutex 并阻塞当前线程  
    atomic {  
        unlock(mutex);  
        block(); // 线程进入睡眠，不再占用 CPU  
    }  
  
    // 当线程被唤醒后，重新获取 mutex（可能再次阻塞）  
    lock(mutex);  
}
```





条件变量

- 条件变量与一个互斥锁（**mutex**）配合使用，互斥锁用于保护共享条件变量所依赖的共享数据。

```
cond_signal(cond) {  
    if (cond->wait_queue 非空) {  
        // 从队列中取出一个等待线程  
        thread = dequeue(cond->wait_queue);  
        // 将该线程变为就绪态（不自动获取锁，需要被唤醒后自己抢锁）  
        wakeup(thread);  
    }  
    // 如果队列为空，信号丢失（无状态）  
}}
```



如何让父线程等待子线程？

```
1  void *child(void *arg) {
2      printf("child\n");
3      // XXX how to indicate we are done?
4      return NULL;
5  }
6
7  int main(int argc, char *argv[]) {
8      printf("parent: begin\n");
9      pthread_t c;
10     Pthread_create(&c, NULL, child, NULL); // create child
11     // XXX how to wait for child?
12     printf("parent: end\n");
13     return 0;
14 }
```

期望的输出

```
parent: begin
child
parent: end
```





基于自旋的解决方案

```
1  volatile int done = 0;
2
3  void *child(void *arg) {
4      printf("child\n");
5      done = 1;
6      return NULL;
7  }
8
9  int main(int argc, char *argv[]) {
10     printf("parent: begin\n");
11     pthread_t c;
12     Pthread_create(&c, NULL, child, NULL); // create child
13     while (done == 0)
14         ; // spin
15     printf("parent: end\n");
16     return 0;
17 }
```





使用条件变量实现父线程等待子线程

```
int done = 0;

pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER; void thr_join() {
pthread_cond_t c = PTHREAD_COND_INITIALIZER;      Pthread_mutex_lock(&m);
                                                    while (done == 0)
                                                    Pthread_cond_wait(&c, &m);
                                                    Pthread_mutex_unlock(&m);
                                                    }

void thr_exit() {
    Pthread_mutex_lock(&m);
    done = 1;
    Pthread_cond_signal(&c);
    Pthread_mutex_unlock(&m);
}

void *child(void *arg) {
    printf("child\n");
    thr_exit();
    return NULL;
}

int main(int argc, char *argv[]) {
    printf("parent: begin\n");
    pthread_t p;
    Pthread_create(&p, NULL, child, NULL);
    thr_join();
    printf("parent: end\n");
    return 0;
}
```

- Pthread_cond_wait(cond,mutex)用于在条件变量上等待，cond是指向条件变量的指针，mutex是指向互斥锁是指针
- 在调用Pthread_cond_wait之前，mutex互斥锁必须被锁定，在函数返回时，则会自动锁定mutetx互斥锁



信号量Semaphore

信号量是另一种临界区的保护机制，它是操作系统提供的一种协调共享资源访问的方法。





信号量

- 信号量是由荷兰科学家Dijkstra提出的，是一种卓有成效的进程同步机制
- 信号量 S 是包含一个整数值的对象，只能通过两个原子操作 `wait()` 和 `signal()` 访问，有时也称为 $P()$ 和 $V()$ 操作

```
wait(S) {  
    while (S <= 0)  
        ; // 忙等待  
    S--;  
}
```

```
signal(S) {  
    S++;  
}
```



基于POSIX标准的信号量

- 信号量的初始化

```
#include <semaphore.h>
```

```
sem_t s;
```

```
sem_init(&s, 0, 1);
```

将信号量初始化为第3个参数

- 对信号量进行操作的函数sem_wait(),sem_post()的定义

```
int sem_wait(sem_t *s) {
```

```
    decrement the value of semaphore s by one
```

```
    wait if value of semaphore s is negative
```

```
}
```

```
int sem_post(sem_t *s) {
```

```
    increment the value of semaphore s by one
```

```
    if there are one or more threads waiting, wake one
```

```
}
```





信号量的实现-不忙等待

- 每个信号量都有一个相关的等待队列
- 每个信号量都有一个整数value

```
#define SEM_NAME_LEN 20
```

```
typedef struct semaphore{  
    char name[SEM_NAME_LEN];  
    int value;  
    struct task_struct *queue;  
} sem_t;
```



wait的实现

- 信号量操作sem_wait定义如下:

```
int sys_sem_wait(sem_t *sem)
{
    cli();
    while( sem->value <= 0 )
        sleep_on(&(sem->queue));
    sem->value--;
    sti();
    return 0;
}
```

- sleep_on() 挂起调用它的进程。





sleep的实现

```
void sleep_on(struct task_struct **p)
{
    struct task_struct *tmp;

    if (!p)
        return;
    if (current == &(init_task.task))
        panic("task[0] trying to sleep");
    tmp = *p;
    *p = current;
    current->state = TASK_UNINTERRUPTIBLE;
    schedule();
    if (tmp)
        tmp->state=0;
}
```





signal的实现

- 信号量操作sem_post定义如下:

```
int sys_sem_post(sem_t *sem)
{
    cli();
    sem->value++;
    if( (sem->value) <= 1)
        wake_up(&(sem->queue));
    sti();
    return 0;
}
```

wake_up(P) 重新启动阻塞进程P的执行





wake的实现

```
void wake_up(struct task_struct **p)
{
    if (p && *p) {
        (**p).state=0;
        *p=NULL;
    }
}
```





二值信号量（锁）

- 将二值信号量作为锁
- 二值信号量的值只能为0和1，也称为互斥锁

```
sem_t m;  
sem_init(&m, 0, X); // initialize semaphore to X; what should X be?  
  
sem_wait(&m);  
// critical section here  
sem_post(&m);
```

单线程使用一个信号量

信号量的值	线程 0	线程 1
1		
1	调用 sem_wait()	
0	sem_wait()返回	
0	（临界区）	
0	调用 sem_post()	
1	sem_post()返回	



两个线程使用一个信号量

值	线程 0	状态	线程 1	状态
1		运行		就绪
1	调用 sem_wait()	运行		就绪
0	sem_wait()返回	运行		就绪
0	(临界区: 开始)	运行		就绪
0	中断; 切换到→T1	就绪		运行
0		就绪	调用 sem_wait()	运行
-1		就绪	sem 减 1	运行
-1		就绪	(sem<0) →睡眠	睡眠
-1		运行	切换到→T0	睡眠
-1	(临界区: 结束)	运行		睡眠
-1	调用 sem_post()	运行		睡眠
0	增加 sem	运行		睡眠
0	唤醒 T1	运行		就绪
0	sem_post()返回	运行		就绪
0	中断; 切换到→T1	就绪		运行
0		就绪	sem_wait()返回	运行
0		就绪	(临界区)	运行
0		就绪	调用 sem_post()	运行
1		就绪	sem_post()返回	运行





信号量用作条件变量

- 一个线程等待条件成立，另一个线程修改条件并发信号给等待线程，从而唤醒等待线程

```
sem_t s;
```

```
void *
```

```
child(void *arg) {
```

```
    printf("child\n");
```

```
    sem_post(&s); // signal here: child is done
```

```
    return NULL;
```

```
}
```

```
int
```

```
main(int argc, char *argv[]) {
```

```
    sem_init(&s, 0, X); // what should X be?
```

```
    printf("parent: begin\n");
```

```
    pthread_t c;
```

```
    Pthread_create(c, NULL, child, NULL);
```

```
    sem_wait(&s); // wait here for child
```

```
    printf("parent: end\n");
```

```
    return 0;
```

```
}
```

父线程等待子线程



信号量用作条件变量

■ 父线程等待子线程（场景1）

值	父线程	状态	子线程	状态
0	create(子线程)	运行	(子线程产生)	就绪
0	调用 sem_wait()	运行		就绪
-1	sem 减 1	运行		就绪
-1	(sem<0)→ 睡眠	睡眠		就绪
-1	切换到→子线程	睡眠	子线程运行	运行
-1		睡眠	调用 sem_post()	运行
0		睡眠	sem 增 1	运行
0		就绪	wake(父线程)	运行
0		就绪	sem_post()返回	运行
0		就绪	中断；切换到→父线程	就绪
0	sem_wait()返回	运行		就绪





信号量用作条件变量

■ 父线程等待子线程（场景2）

值	父线程	状态	子线程	状态
0	create（子线程）	运行	(子线程产生)	就绪
0	中断；切换到→子线程	就绪	子线程运行	运行
0		就绪	调用 sem_post()	运行
1		睡眠	sem 增 1	运行
1		就绪	wake(没有线程)	运行
1		就绪	sem_post()返回	运行
1	父线程运行	运行	中断；切换到→父线程	就绪
1	调用 sem_wait()	运行		就绪
0	sem 减 1	运行		就绪
0	(sem>=0)→不用睡眠	运行		就绪
0	sem_wait()返回	运行		就绪





计数（记录）信号量

- 信号量中的**value**表示资源的可用数目。具体来说：
 - $\text{value} > 0$: 表示有 value 个资源可用，其值即为当前可用资源的数量。
 - $\text{value} = 0$: 表示没有资源可用。
 - $\text{value} < 0$: 表示有 $|\text{value}|$ 个进程正在等待该资源， $|\text{value}|$ 的绝对值表示等待该资源的进程数。



生产者/消费者(有界缓冲区)问题

- 问题描述：假设有一个或多个生产者线程和一个或多个消费者线程。生产者把生成的数据项放入缓冲区;消费者从缓冲区取走数据项，以某种方式消费。
- 应用场景：
 - 在多线程的网络服务器中，一个生产者将 HTTP 请求放入工作队列(即有界缓冲区)，消费线程从队列中取走请求并处理。
 - 管道指令：`grep foo file.txt | wc -l`，grep进程是生产者，wc进程是消费者





信号量解决生产者/消费者问题

```
int buffer[MAX];
int fill = 0;
int use = 0;

void put(int value) {
    buffer[fill] = value;    // Line F1
    fill = (fill + 1) % MAX; // Line F2
}

int get() {
    int tmp = buffer[use];    // Line G1
    use = (use + 1) % MAX;    // Line G2
    return tmp;
}
```

```
int main(int argc, char *argv[]) {
    // ...
    sem_init(&empty, 0, MAX); // MAX are empty
    sem_init(&full, 0, 0);    // 0 are full
    // ...
}
```

```
sem_t empty; //缓冲区空
sem_t full;  //缓冲区满

void *producer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        sem_wait(&empty);    // Line P1
        put(i);              // Line P2
        sem_post(&full);     // Line P3
    }
}

void *consumer(void *arg) {
    int tmp = 0;
    while (tmp != -1) {
        sem_wait(&full);     // Line C1
        tmp = get();         // Line C2
        sem_post(&empty);    // Line C3
        printf("%d\n", tmp);
    }
}
```

当 $MAX > 1$ 时，哪里产生了竞态条件？





生产者/消费者问题

1. 互斥锁 (Mutex)：用于保证同一时间只有一个进程/线程访问共享资源。
2. 条件变量 (Condition Variable)：用于进程/线程间的等待和通知。
 - 消费者条件：当缓冲区空时，消费者等待。
 - 生产者条件：当缓冲区满时，生产者等待。

使用三个信号量：

- Mutex：表示互斥锁，初始值为1
- empty：表示缓冲区中空余空间的数量。
- full：表示缓冲区中已占用空间的数量。





生产者/消费者问题

■ 增加互斥的方案

⇒ 死锁

```
void *producer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        sem_wait(&mutex);           // Line P0 (NEW LINE)
        sem_wait(&empty);           // Line P1
        put(i);                     // Line P2
        sem_post(&full);             // Line P3
        sem_post(&mutex);           // Line P4 (NEW LINE)
    }
}

void *consumer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        sem_wait(&mutex);           // Line C0 (NEW LINE)
        sem_wait(&full);            // Line C1
        int tmp = get();            // Line C2
        sem_post(&empty);           // Line C3
        sem_post(&mutex);           // Line C4 (NEW LINE)
        printf("%d\n", tmp);
    }
}
```



生产者/消费者问题

■ 避免死锁：减少锁的作用域

```
void *producer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        sem_wait(&empty);           // Line P1
        sem_wait(&mutex);           // Line P1.5 (MUTEX HERE)
        put(i);                     // Line P2
        sem_post(&mutex);           // Line P2.5 (AND HERE)
        sem_post(&full);            // Line P3
    }
}

void *consumer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        sem_wait(&full);            // Line C1
        sem_wait(&mutex);           // Line C1.5 (MUTEX HERE)
        int tmp = get();            // Line C2
        sem_post(&mutex);           // Line C2.5 (AND HERE)
        sem_post(&empty);           // Line C3
        printf("%d\n", tmp);
    }
}
```



读者-写者问题

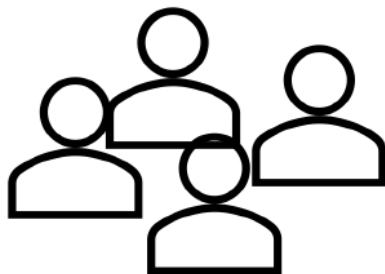
公告栏问题

这个公告栏
要撤走了



写者

公告栏



读者

别挤，再挤
就看不到了

思考：多个读者如果希望读公告栏，他们互斥吗？

思考：如何避免读者看到一半就被写者撤走了，我们怎么办？





读者-写者问题

- 一个数据集在多个并发进程之间共享
 - 读者 — 仅读取数据集；他们不执行任何更新
 - 写者 — 可以读写
 - 允许多个读者同时阅读
 - 只有一个写者可以同时访问共享数据
- 读者和写者的几种变体都涉及某种形式的**优先级**
 - 读者优先：当写者提出存取共享数据集的要求后，仍允许新读者进入。**更好的并行性**
 - 写者优先：当写者提出存取共享数据集的要求后，不允许新读者进入，且等待的写者可以跳过等待的读者进行写操作。**更加公平**



读者-写者问题

1. 允许多个读者同时执行读操作。
2. 不允许读者、写者同时操作。
3. 不允许多个写者同时操作。





读者优先

思想：只要有读者在读，新来的读者直接进入，写者必须等待所有读者完成。

使用二个信号量和计数值：

- write_mutex：写互斥锁，确保只有一个写者能进入临界区，初值为1。
- readers：临界区读者的数量，初值为0。
- mutex：互斥锁，用于互斥访问readers，初值为1。





读者优先

读者:

```
while (1) {  
    wait(mutex);  
    read_count++;  
    if (read_count == 1)  
        wait(write_mutex);  
    signal(mutex);  
    // 执行读操作  
    wait(mutex);  
    read_count--;  
    if (read_count == 0)  
        signal(write_mutex);  
    signal(mutex);  
}
```

写者:

```
while (true) {  
    wait(write_mutex);  
    // 执行写操作  
    signal(write_mutex);  
}
```





写者优先

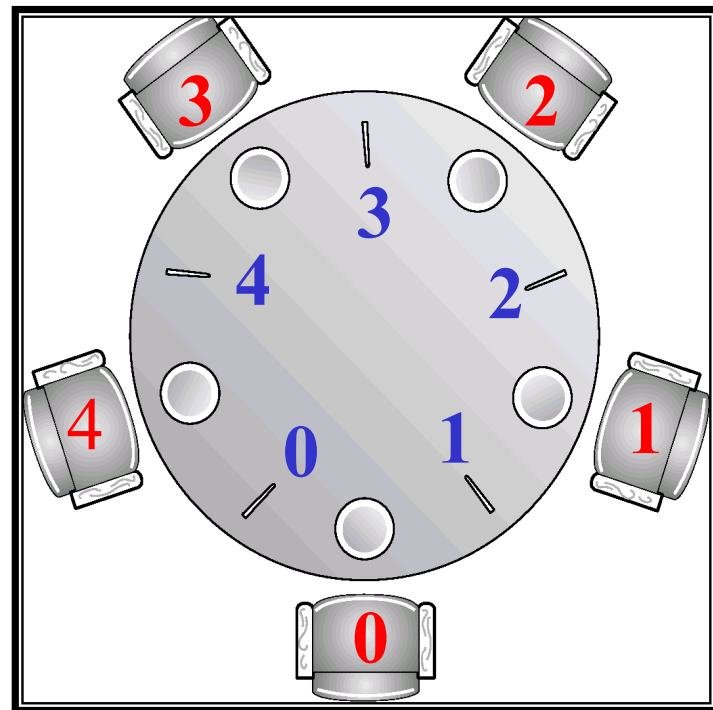
思想：写者到达后，阻止新读者进入（等待已有读者完成），写者优先于后续读者。

使用三个信号量和计数值：

- `write_mutex`：写互斥锁，确保只有一个写者能进入临界区，初值为1。
- `readers`：临界区读者的数量，初值为0。
- `mutex`：互斥锁，用于互斥访问`readers`，初值为1。
- `write_wait`：控制读者进入，写者等待时阻塞新读者，初值为1。

哲学家进餐问题

- 假定有 5 位“哲学家”围着一个圆桌。每两位哲学家之间有一根筷子(一共 5 根), 每个哲学家面前有一个碗。
- 哲学家的生活方式是交替地思考和就餐。
- 平时哲学家进行思考, 饥饿时便试图取其左、右最靠近他的筷子, 只有在他拿到两支筷子时才能进餐
- 进餐完毕, 放下筷子又继续思考





哲学家进餐问题

■ 哲学家的基本代码：

```
while (1) {  
    think();  
    get_forks(p);  
    eat();  
    put_forks(p);  
}
```

■ 面临的同步问题：

- 互斥：每根筷子只能被一个人拿起
- 死锁：没人能拿到足够的筷子
- 饥饿：有人总也拿不到筷子
- 如何实现getforks()和putforks()，确保没有死锁，没有哲学家饥饿，且并发度更高（更多的哲学家同时吃东西）





哲学家进餐问题

- 信号量的定义
 - `stick[5]`: 五支筷子的互斥访问权, 初值为1
- 尝试书写伪代码

第*i*位哲学家:

```
while (true) {  
    wait(stick[i]);  
    wait(stick[(i+1)%5]);  
    // 吃饭  
    signal(stick[i]);  
    signal(stick[(i+1)%5]);  
    // 思考  
};
```

问题: 可能造成死锁





哲学家进餐问题

- 最多允许 4 个哲学家同时进餐
 - number: 计数信号量, 初值为4
- 尝试书写伪代码

第i位哲学家:

```
while (true) {  
    wait (number)  
    wait(stick[i]);  
    wait(stick[(i+1)%5]);  
    // 吃饭  
    signal(stick[i]);  
    signal(stick[(i+1)%5]);  
    signal (number)  
    // 思考  
};
```





其它解决方案

- 下面列出了多个可能解决死锁问题的方法：
 - 修改最后一个哲学家取筷子的顺序。
 - 仅当左、右两支筷子均可用时，才允许拿起筷子进餐。
 - 奇数号哲学家先拿左筷子再拿右筷子，偶数号哲学家相反。





睡眠的理发师问题

- 理发店里有一位理发师，一把理发椅和N把供等候理发顾客坐的椅子。
- 如果没有顾客，理发师睡眠，当一个顾客到来时叫醒理发师；
- 若理发师正在理发时又有顾客来，那么有空椅子就坐下，否则离开理发店。





睡眠的理发师问题

- 定义三个信号量：
 - customers记录等候理发的顾客数（不包括正在理发的顾客）；初值为0
 - barbers用于表示理发师是否可用；初值为0
 - mutex用于互斥。初值为1
 - 变量count记录等候的顾客数，它是customers的一份拷贝。之所以使用count是因为无法读取信号量的当前值。





理发师和顾客的描述

```
barber(){  
    while(true){  
        wait(customers); //是否有等候的顾客  
        wait(mutex);  
        count=count-1; // 顾客数减1  
        signal(mutex);  
        signal(barbers); // 理发师开始理发  
        理发;  
    }  
}
```

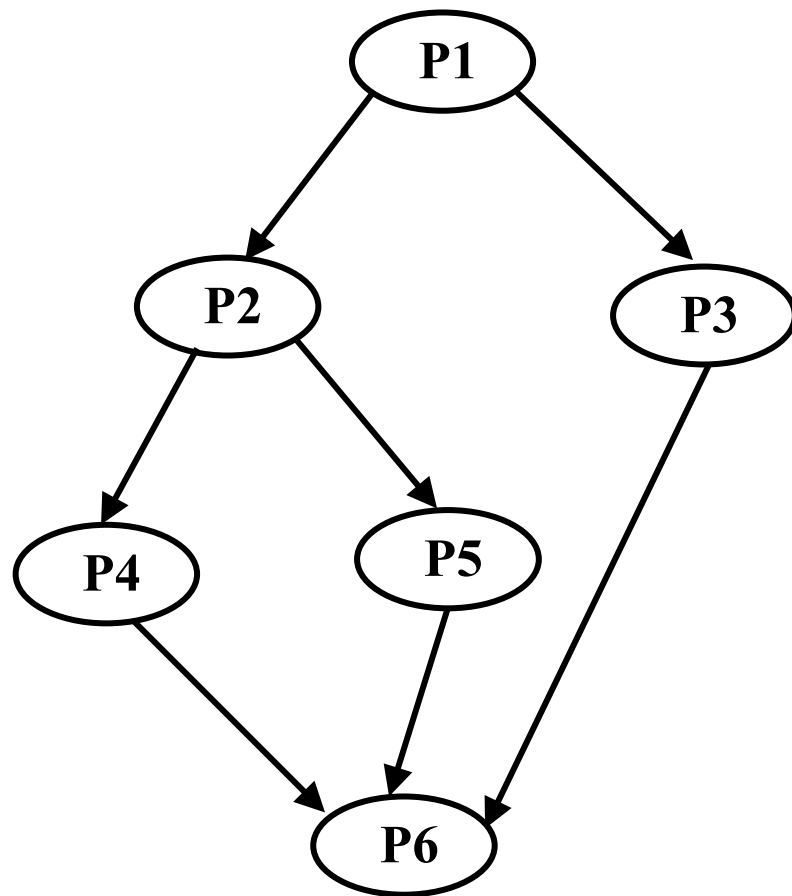
```
customer(){  
    wait(mutex);  
    if(count < N) {  
        count=count+1; // 等候的顾客加1  
        signal(mutex);  
        signal(customers); //通知理发师有顾客  
        wait(barbers); // 等待理发师准备好  
        理发;  
    }  
    else    signal(mutex);  
}
```





利用信号量实现进程间的同步

- P1、P2、P3、P4、P5、P6为一组合作进程，其前驱图如下所示，试用信号量实现这六个进程间的同步。





解法1

- 设七个同步信号量a、b、c、d、e、f、g分别表示进程之间的前驱关系，如图所示，其初值均为0。

P1 ()

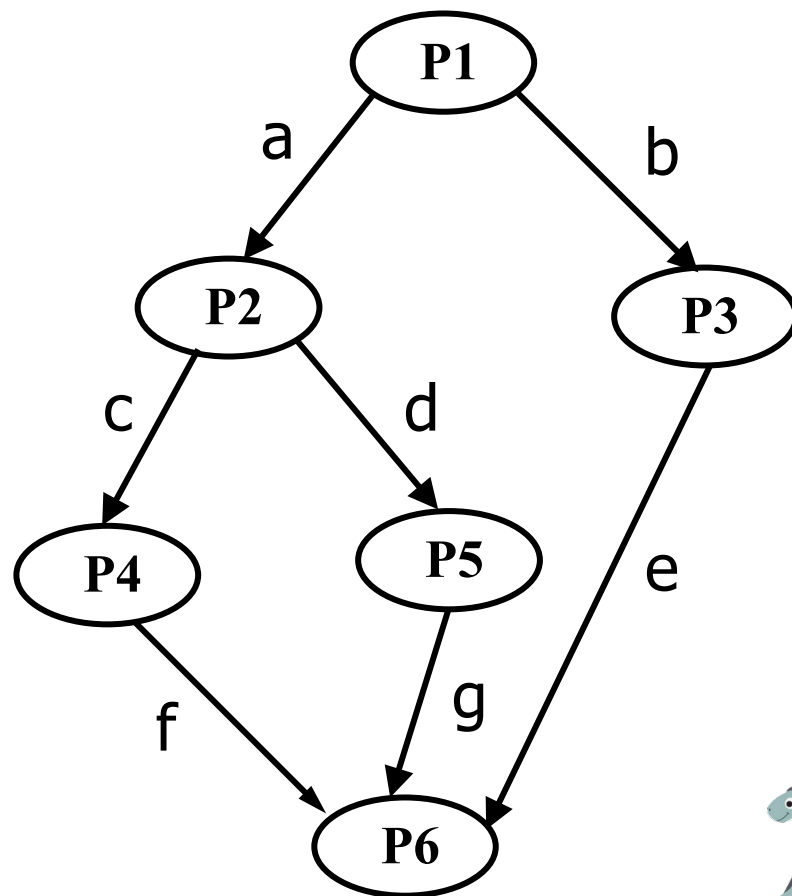
{

 执行P1的代码;

 v(a);

 v(b);

}





解法2

- 设五个同步信号量f1、f2、f3、f4、f5分别表示进程P1、P2、P3、P4、P5是否执行完成，其初值均为0。

P1 ()

{

执行P1的代码;

v(f1);

v(f1);

}

P2 ()

{

p(f1);

执行P2的代码;

v(f2);

v(f2);

}

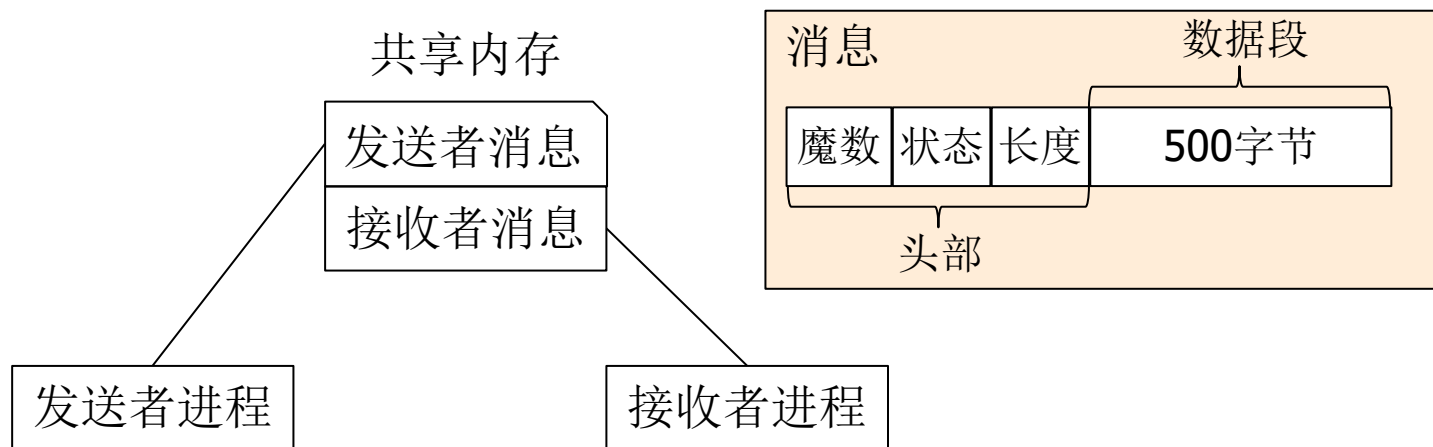


进程间通信

- 通过多进程间的协作来实现应用和系统是一种广泛采纳的开发方法
- 多进程协作的优势
 - 将功能模块化，避免重复造轮子
 - 增强模块间的隔离，提供更强的安全保障
 - 提高应用的容错能力
- 进程间通信(Inter-Process Communicatiton, **IPC**)是多进程协作的基础
- 进程通信机制：共享内存、消息传递、管道



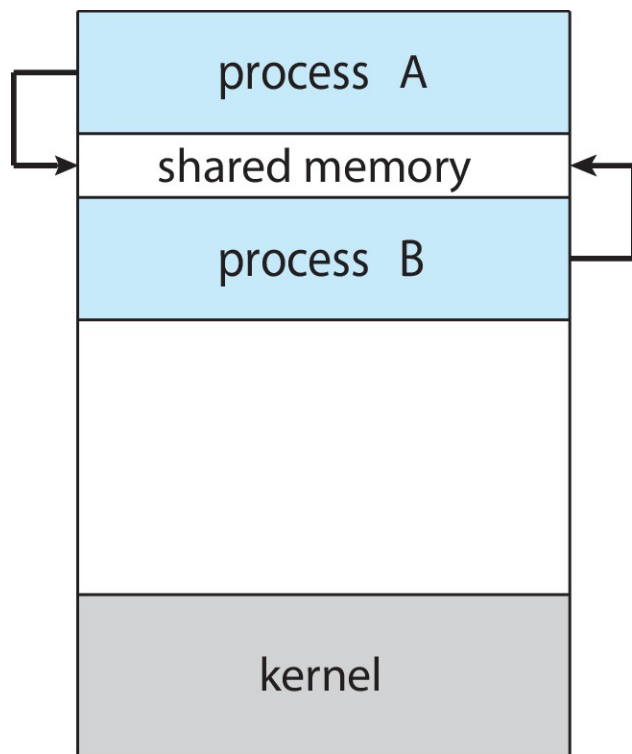
一种进程间的通信设计



简单的进程间通信设计

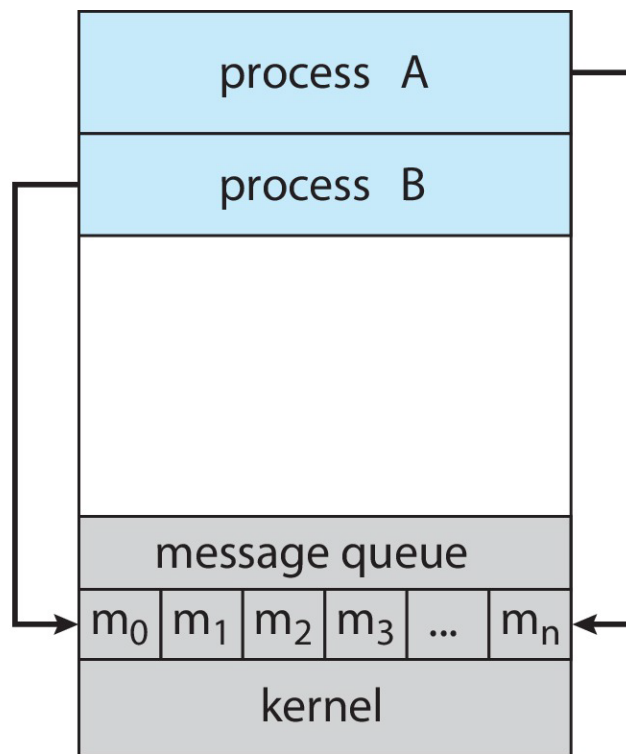
- 发送者进程和接收者进程彼此隔离
- 通过共享内存进行通信
- 发送者发送消息，接收者接收消息

进程通信的两种模型



(a)

共享内存



(b)

消息传递

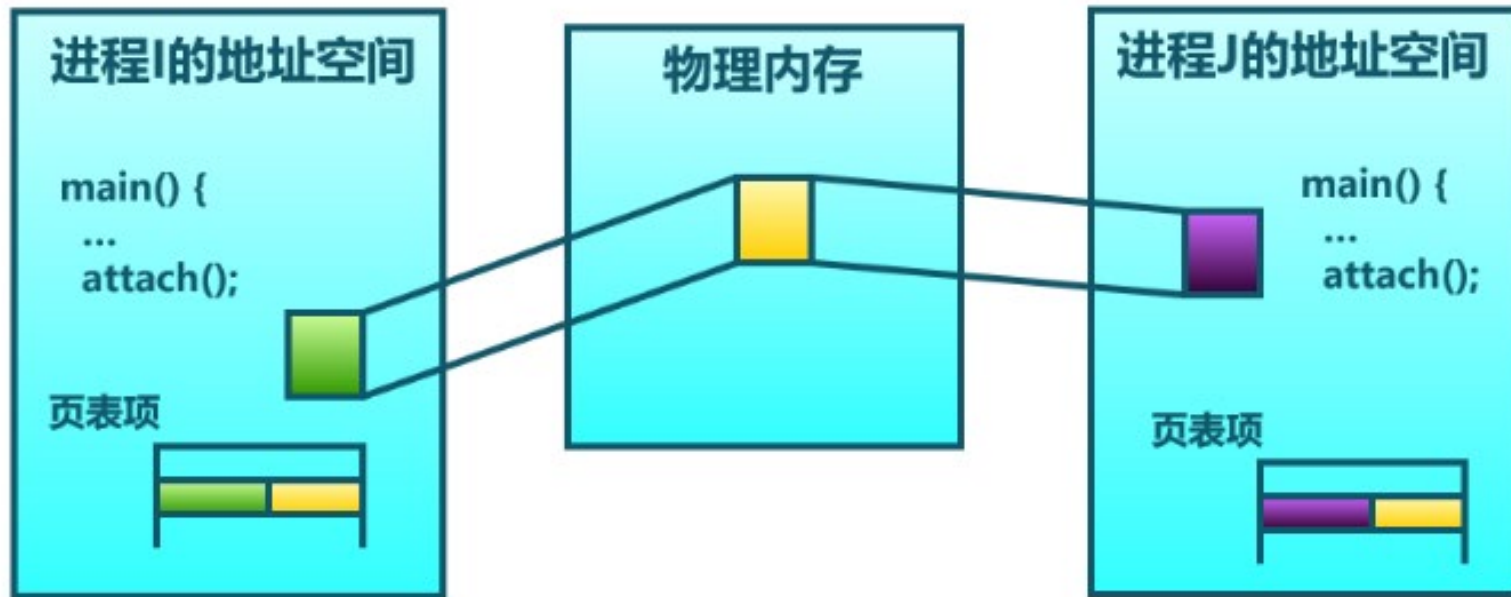


共享内存机制

- 共享内存（Shared Memory）是一种最快捷的进程间通信方法，它允许多个进程访问同一块物理内存空间。这种通信方式非常高效，因为数据不需要在进程之间复制。
- 基本原理：
 - 内核为需要通信的进程建立共享区域
 - 参与通信的多方进程将共享区域附加到它们自己的地址空间。所有进程都可以访问共享内存中的地址，就像访问自己的私有内存一样。



共享内存机制

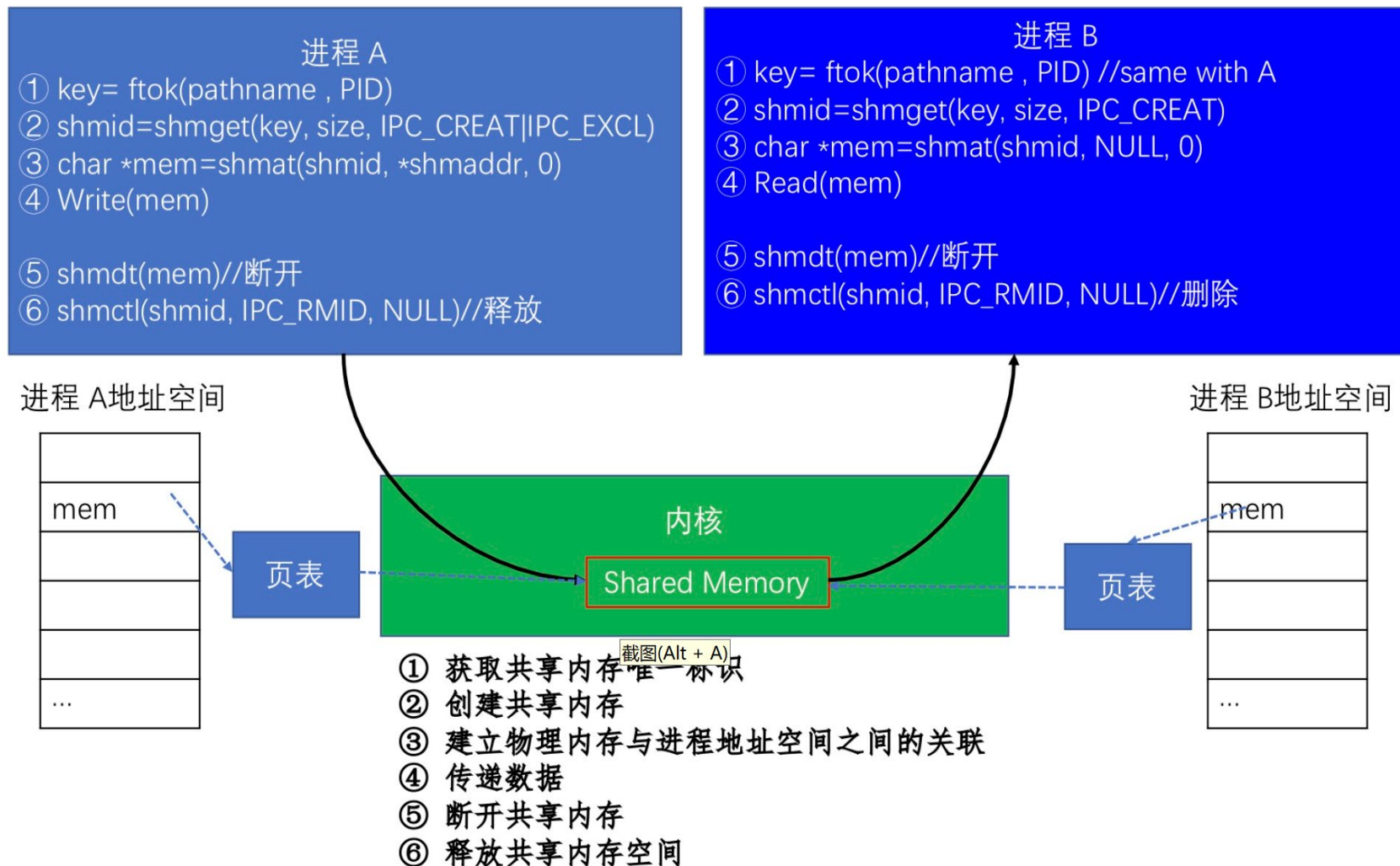




Linux的共享内存机制

- Linux系统中，共享内存的创建和控制主要通过以下几个系统调用实现：
 - **shmget**：创建一个新的共享内存段或者获取已存在的共享内存段。此函数需要指定一个键值和共享内存的大小。
 - **shmat**：将共享内存段附加到当前进程的地址空间。此函数需要指定shmget返回的共享内存标识符。
 - **shmdt**：将共享内存段从当前进程的地址空间中分离。
 - **shmctl**：允许你对共享内存进行各种控制操作，如设置权限、删除共享内存等。
- 当多个进程同时访问和修改共享内存时，需要利用信号量或互斥锁确保数据的一致性。

共享内存实现机制





reader

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#define BUFSZ 512
int main(int argc, char *argv[])
{
    int shmid;
    int ret;
    key_t key;
    char *shmadd;

    //创建key值
    key = ftok("../", 2015);
    if(key == -1)
    {
        perror("ftok");
    }

    system("ipcs -m"); //查看共享内存

    //打开共享内存
    shmid = shmget(key, BUFSZ, IPC_CREAT|0666);
    if(shmid < 0)
    {
        perror("shmget");
        exit(-1);
    }
}
```

//映射

```
shmadd = shmat(shmid, NULL, 0);
if(shmadd < 0)
{
    perror("shmat");
    exit(-1);
}

//读共享内存区数据
printf("data = [%s]\n", shmadd);

//分离共享内存和当前进程
ret = shmdt(shmadd);
if(ret < 0)
{
    perror("shmdt");
    exit(1);
}
else
{
    printf("deleted
shared-memory\n");
}

//删除共享内存
shmctl(shmid, IPC_RMID, NULL);

system("ipcs -m"); //查看共享内存

return 0;
}
```





writer

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#define BUFSZ 512
int main(int argc, char *argv[])
{
    int shmid;
    int ret;
    key_t key;
    char *shmadd;

    //创建key值
    key = ftok("../", 2015);
    if(key == -1)
    {
        perror("ftok");
    }

    //创建共享内存
    shmid = shmget(key, BUFSZ, IPC_CREAT | 0666);
```

```
if(shmid < 0)
{
    perror("shmget");
    exit(-1);
}

//映射
shmadd = shmat(shmid, NULL, 0);
if(shmadd < 0)
{
    perror("shmat");
    _exit(-1);
}

//拷贝数据至共享内存区
printf("Writer: copy data to shared-
memory\n");

bzero(shmadd, BUFSZ); // 共享内存清空
strcpy(shmadd, " How are you, mike: from
Writer ");

return 0;
}
```





消息传递机制

- 消息传递（Message Passing）是一种基本的进程间通信（IPC）机制，它允许进程之间通过发送和接收消息来交换信息。这种机制通常由操作系统内核提供支持，并通过系统调用或库函数向应用程序提供服务。
- 分类：
 - 直接通信方式：发送进程将消息发送到接收进程，并将其挂在接收进程的消息队列上；接收进程从消息队列上取消息。
 - 间接通信方式：发送进程将消息发送到信箱，接收进程从信箱中取消息。



消息缓冲通信

- 消息缓冲通信是直接通信方式的一种实现
- 消息缓冲区的数据结构

```
struct message {
```

```
    sender; 发送者进程标识符
```

```
    size; 消息长度
```

```
    text; 消息正文
```

```
    next; 指向下一个消息缓冲区的指针
```

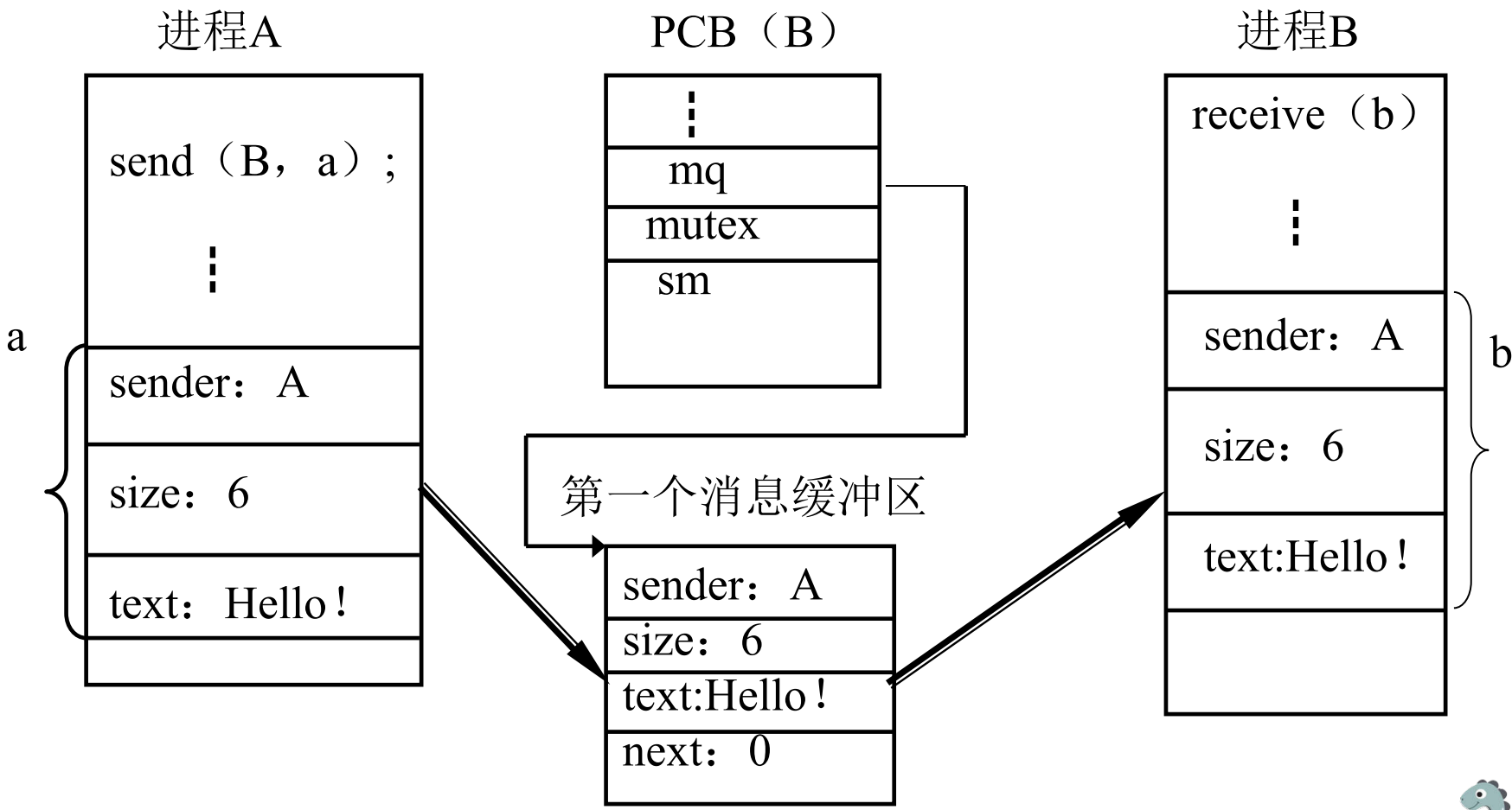
```
}
```

- PCB中需增加的内容
 - mq;消息队列队首指针
 - mutex;消息队列互斥信号量
 - sm;消息队列资源信号量





两个进程进行通信的过程





发送原语描述

```
void send (receiver, a)
```

```
/*receiver为接收者标识号，a为发送区首址*/
```

```
{
```

```
    向系统申请一个消息缓冲区i;
```

```
    将发送区a中的消息复制到i中;
```

```
    获得接收进程的內部标识j;
```

```
    P (mutex) ;
```

```
    把消息插入j的消息队列上;
```

```
    V (mutex) ;
```

```
    V (sm) ;
```

```
}
```





接收原语描述

```
void receive (b) /*b为接收区首址*/  
{  
    获得接收进程内部标识j;  
    P (sm) ;  
    P (mutex) ;  
    将消息队列中的第一个消息移出;  
    V (mutex) ;  
    将消息复制到接收区b;  
}
```



Linux的消息队列

■ 消息队列的系统调用

- `msgget ( key, flags )` //获取消息队列标识
- `msgsnd ( QID, buf, size, flags )` //发送消息
- `msgrcv ( QID, buf, size, type, flags )` //接收消息
- `msgctl( ... )` // 消息队列控制

消息的结构

```
struct msgbuf {  
    long mtype; /* 消息的类型 */  
    char mtext[1]; /* 消息正文 */  
};
```



消息队列实现机制

进程 A

- ① `key= ftok(pathname , PID)`
- ② `msgid = msgget(key, IPC_CREAT | IPC_EXCL | 0600)`
- ③ `msgsnd(msgid, &buf, sizeof(buf), flag)`
- ④ `msgctl(msqid, IPC_RMID, NULL)//删除`

进程 B

- ① `key= ftok(pathname , PID)`
- ② `msgid = msgget(key, IPC_CREAT | 0600)`
- ③ `msgrcv(msgid, &buf, sizeof(buf), type, flag)`
- ④ `msgctl(msqid, IPC_RMID, NULL)//删除`

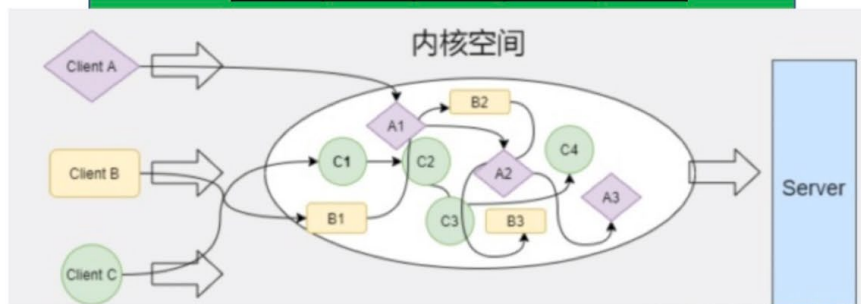
sys_ipc

内核

消息队列: 消息转发

m₀ m₁ m₂ m_n

- ① 获取唯一标识
- ② 创建消息队列
- 内核中转消息
- ③ 收发消息
- ④ 删除消息队列





信箱通信

- 信箱通信方式中，进程之间通信需要通过共享数据结构实体——信箱来进行。
- 信箱是一种数据结构，其中存放信件。
- 信箱逻辑上分成信箱头和信箱体两部分。
 - 信箱头中存放有关信箱的描述。
 - 信箱体由若干格子组成，每格存放一个信件，格子的数目和大小在创建信箱时确定。
- 信箱通信原语包括：
 - 信箱的创建和撤消
 - 消息的发送和接收：
 - `Send(mailbox, message);`
 - `Receive(mailbox, message);`





消息通信中的同步问题

- 进程间的消息通信存在同步关系
- 对于发送进程来说，它在执行发送原语后有两种可能选择：
 - 发送进程阻塞，直到这个消息被接收进程接收到，这种发送称为阻塞发送。
 - 发送进程不阻塞，继续执行，这种发送称为非阻塞发送。



消息通信中的同步问题

- 对于一个接收进程来说，在执行接收原语后也有两种可能选择：
 - 如果一个消息在接收原语执行之前已经发送，则该消息被接收进程接收，接收进程继续执行。
 - 如果没有正在等待的消息，则该进程阻塞直到有消息到达；或者该进程继续执行，放弃接收的努力。前者称为阻塞接收，后者称为非阻塞接收。



消息通信中的同步问题

- 根据发送进程和接收进程采取方式的不同，通常有三种常用的组合方式：
 - 非阻塞发送、阻塞接收。
 - 非阻塞发送、非阻塞接收。
 - 阻塞发送，阻塞接收。



管道通信机制

- 管道（pipe）在Unix系列的系统中会被当作一个文件，内核会为用户态提供代表管道的文件描述符，让其可以通过文件相关系统调用来使用。
- 管道不会使用存储设备，而是使用内存作为数据的缓冲区。
- 主要用于父子进程间或者兄弟进程间的通信。它允许一个进程向另一个进程传递数据。
- 管道有两种类型：无名管道（pipe）和有名管道（named pipe，也叫FIFO）。



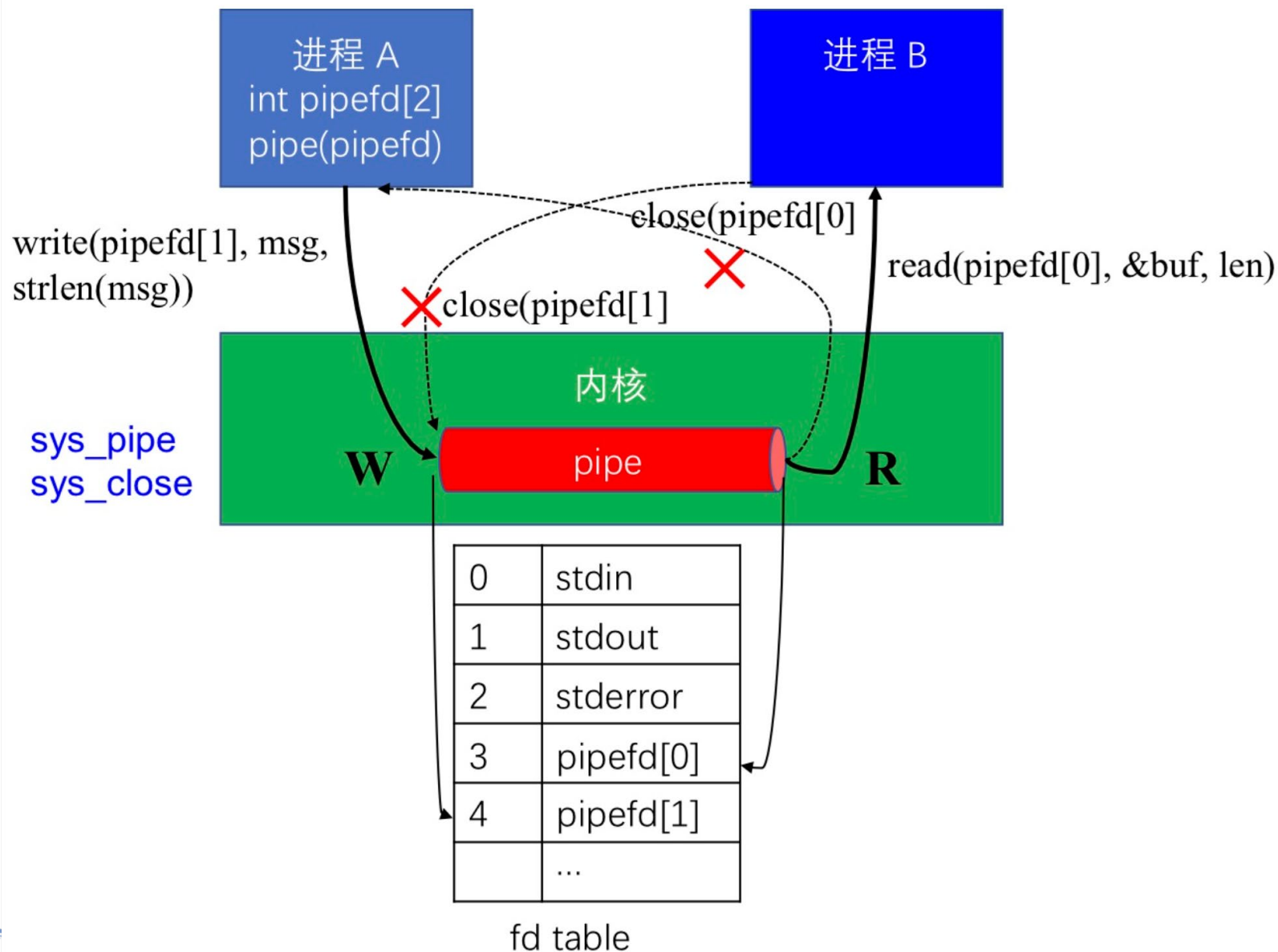


管道通信机制

- **无名（匿名）管道**主要用于父子进程间的通信，其基本原理是：在创建管道时，系统返回两个文件描述符，分别代表管道的读端和写端。父进程可以通过写端向管道中写入数据，子进程则可以通过读端从管道中读取数据。
- **有名（命名）管道（FIFO）**则可以用于任何两个进程间的通信。它是一种特殊的文件，可以像普通文件一样用open、read、write等系统调用进行操作。一个进程向FIFO中写入的数据可以被另一个进程读取出来。

- 使用管道通信时，基本上采用文件系统的原有机制实现。包括创建、打开、关闭、读写等。
- 管道机制应提供以下三方面的协调能力：
 - 互斥：诸进程互斥读写管道
 - 同步：管道空、满情况处理
 - 存在：确定对方是否存在

无名管道实现机制





无名管道示例

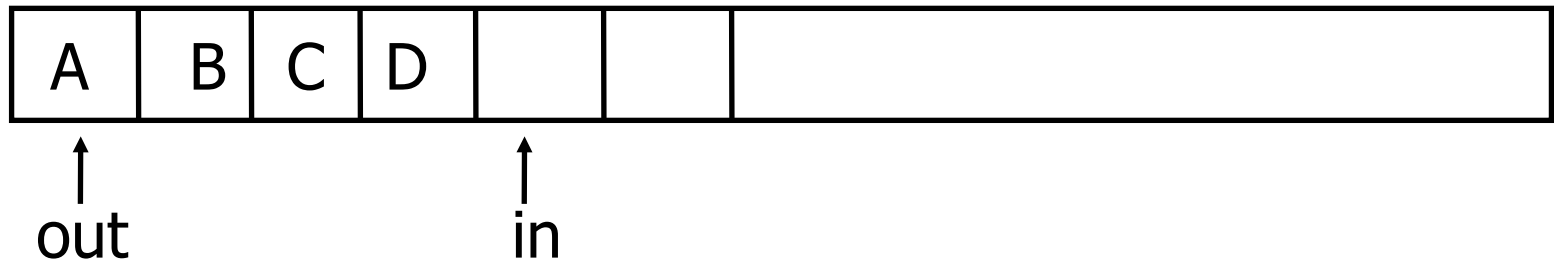
```
$ gcc -o ex1 ex1.c
$ ./ex1
parent
write: the 0 message.
write: the 1 message.
...
children
read: the 0 message.
read: the 1 message.
...
```

```
2  #include <stdio.h>
3  #include <unistd.h>
4  #include <string.h>
5  #include <sys/wait.h>
6  int main()
7  {
8      __pid_t pid;
9      int fd[2];
10     char str[20];
11     pipe(fd);
12     if ((pid = fork() == 0))
13     {
14         printf("children\n");
15         close(fd[1]);
16         int num;
17         while ((num = read(fd[0], str, sizeof(str)) != 0))
18         {
19             printf("read: %s", str);
20         }
21     }
22     else
23     {
24         printf("parent\n");
25         close(fd[0]);
26         for (int i = 0; i < 10; i++)
27         {
28             sprintf(str, "the %d message. \n", i);
29             printf("write: %s", str);
30             write(fd[1], str, sizeof(str));
31         }
32     }
33 }
```



管道通信示意图

- 初始时，其长度为4，系统将管道看成一个循环队列。按先进先出的方式读写。



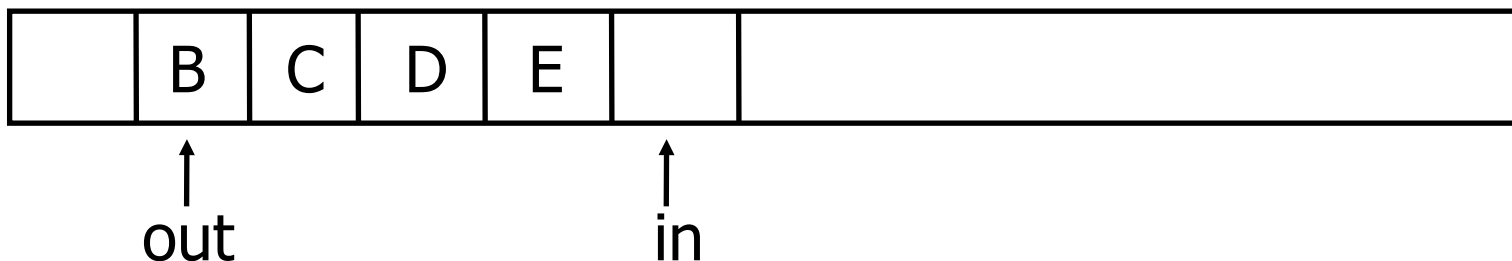
- 写入字符E后，管道长度为5



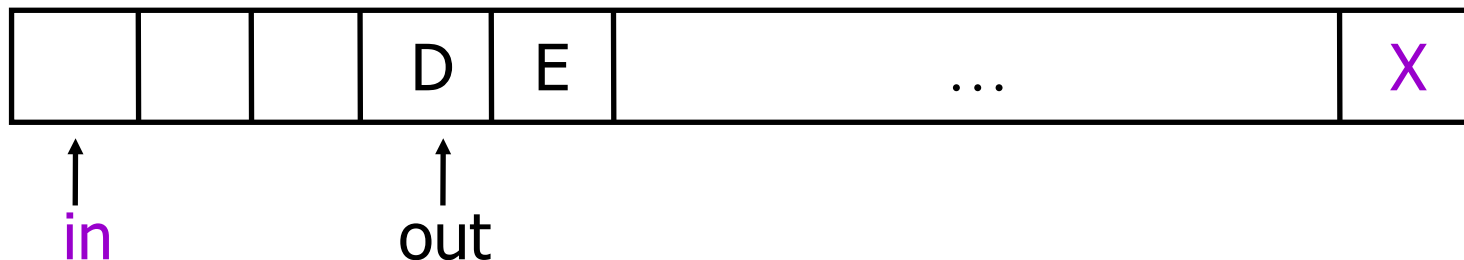


管道通信示意图

- 读一个字符后，管道长度为4



- 若管道容量为 n 且 $in=n$ 时，再写入一个字符，则 in 移到管道的另一端。



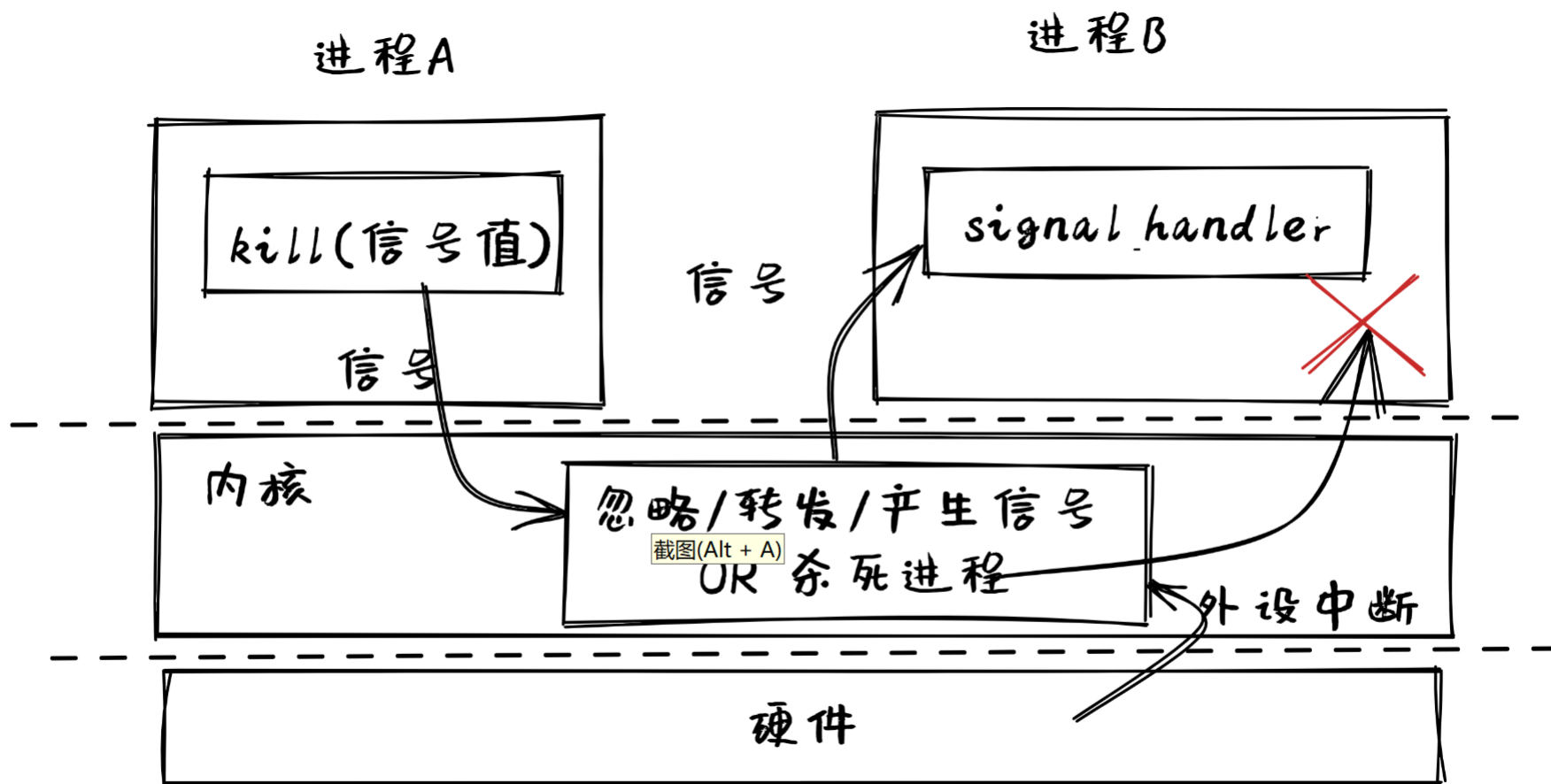


信号 (signal)

- 信号是中断正在运行的进程的异步消息或事件
- 信号机制是一种进程间异步通知机制
- 信号是一个整数编号，这些整数编号都定义了对应的宏名，宏名都是以SIG开头，比如SIGABRT, SIGKILL, SIGSTOP, SIGCONT
- Linux有62个信号



信号实现机制





练习题

- 6.6 Race conditions are possible in many computer systems. Consider a banking system that maintains an account balance with two functions: `deposit(amount)` and `withdraw(amount)`. These two functions are passed the amount that is to be deposited or withdrawn from the bank account balance. Assume that a husband and wife share a bank account. Concurrently, the husband calls the `withdraw()` function, and the wife calls `deposit()`. Describe how a race condition is possible and what might be done to prevent the race condition from occurring.
- 6.23 Servers can be designed to limit the number of open connections. For example, a server may wish to have only N socket connections at any point in time. As soon as N connections are made, the server will not accept another incoming connection until an existing connection is released. Illustrate how semaphores can be used by a server to limit the number of concurrent connections.





选择题1

- 若信号量S的初值为2，当前值为-1，则表示有_____等待进程。
 - A. 2个 B. 1个
 - C. 0个 D. 3个
- 在操作系统中，P、V操作是一种_____。
 - A. 机器指令 B. 系统调用命令
 - C. 作业控制命令 D. 低级进程通信原语





选择题2

- 下述哪个选项不是管程的组成部分_____。
 - A.对局部于管程的数据结构设置初值的语句
 - B.局部于管程的共享数据说明
 - C.管程内对数据结构进行操作的一组过程
 - D.管程外过程调用管程内数据结构的说明
- 临界区是_____。
 - A.一段共享数据区 B.一段程序
 - C.一个互斥资源 D.一个缓冲区





选择题3

- 用P、V操作管理临界区时，信号量的初值应定义为_____。
 - A. 1 B. 2
 - C. -1 D. 0
- 对于两个并发进程，设互斥信号量为mutex,若mutex=0则_____。
 - A.表示有一个进程进入临界区，另一个进程等待进入
 - B. 表示没有进程进入临界区
 - C.表示有一个进程进入临界区
 - D. 表示有两个进程进入临界区





选择题4

- 对信号量S执行V操作后，下述选项正确的是_____。
 - A. 当S小于0时唤醒一个阻塞进程
 - B. 当S小于等于0时唤醒一个阻塞进程
 - C. 当S小于0时唤醒一个就绪进程
 - D. 当S小于等于0时唤醒一个就绪进程
- 对信号量X执行P操作时，若_____则进程进入等待状态。
 - A. $X-1 < 0$
 - B. $X-1 \leq 0$
 - C. $X-1 > 0$
 - D. $X-1 \geq 0$





选择题5

- 有若干并发进程均将共享变量count的值加1一次，那么有关count值说法正确的是_____。
 - A. 得到的结果肯定不正确
 - B. 得到的结果肯定正确
 - C. 若控制这些并发进程互斥执行count加1操作，count中的值正确
 - D. A, B, C均不对





选择题6

- 下述关于管程的描述中错误的是_____。
 - A. 管程是一种进程同步工具，解决了信号量机制中大量同步操作分散问题
 - B. 管程每次只允许一个进程进入管程
 - C. 管程中的signal操作的作用和信号量机制中的signal操作相同
 - D. 管程是被进程调用的





填空题1

- 如果信号量的当前值为-4，则表示系统中在该信号量上有 _____ 个等待进程。
- 对于信号量可以做 _____ ① _____ 操作和 _____ ② _____ 操作，
_____ ③ _____ 操作用于阻塞进程，_____ ④ _____ 操作用于释放进程。程序中的 _____ ⑤ _____ 操作应谨慎使用，以保证其使用的正确性，否则执行时可能发生死锁。
- 信号量的物理意义是：当信号量值大于0时表示 _____ ① _____ ；当信号量值小于0时，其绝对值为 _____ ② _____ 。





填空题2

- 有 m 个进程共享同一临界资源，若使用信号量机制实现对临界资源的互斥访问，则信号量值的变化范围是_____。
- 访问临界资源的进程应该遵循的条件有：
①_____、②_____、③_____、和④_____。
- 临界资源是指_____的资源。
- 管程由①_____、②_____和③_____三部分组成





考研题-1

- 三个进程P1、P2、P3互斥使用一个包含 $N(N>0)$ 个单元的缓冲区，
- P1每次用`produce()`生成一个正整数并用`put()`送入缓冲区某一空单元中；
- P2每次用`getodd()`从该缓冲区中取出一个奇数并用`countodd()`统计奇数个数；
- P3每次用`geteven()`从该缓冲区中取出一个偶数并用`counteven()`统计偶数个数。
- 请用信号量机制实现这三个进程的同步与互斥活动，并说明所定义的信号量含义。要求用伪代码编写。 09





考研题-1答案

```
semaphore empty=N; semaphore odd=0,even=0;
semaphore mutex=1;
main ( )
cobegin
{ Process P1()
  while(true)
  { number=produce();
    P(empty);
    P(mutex);
    put();
    V(mutex);
    if (number % 2 = 0) V(even);
    else V(odd);
  }
}
```





考研题-1答案续

```
Process P2()
while(true)
{ P(odd);
  P(mutex);
  getodd();
  V(mutex);
  V(empty);
  countodd(); }
```

```
Process P3()
while(true)
{ P(even);
  P(mutex);
  geteven();
  V(mutex);
  V(empty);
  counteven();
} } coend
```





考研题2

- 单处理机系统中，可并行的是____。 09
I. 进程与进程 II. 处理机与设备
III. 处理机与通道 IV. 设备与设备
A. I、II和III B. I、II和IV
C. I、III和IV D. II、III和IV
- 设与某资源相关联的信号量初值为3，当前值为1，若M表示该资源的可用个数，N表示等待该资源的进程数，则M、N分别是____。 10
A. 0, 1 B. 1, 0
C. 1, 2 D. 2, 0





考研题3

进程P0和P1的共享变量定义及初值为：

```
boolean flag[2];   int turn=0;
flag[0]=false;flag[1]=false;
```

进程P0和P1访问临界资源的类C代码实现如下：

```
void P0 //进程P0
{ while (TRUE)
  { flag[0]=TRUE;
    turn=1;
    while (flag[1] && turn==1);
      临界区;
    flag[0]=FALSE;
  } }
```

```
void P1 //进程P1
{ while (TRUE)
  { flag[1]=TRUE;
    turn=0;
```

```
while (flag[0] && turn==0);
  临界区;
  flag[1]=FALSE;
} }
```

则并发执行进程P0和P1时产生的情况是_____。

- A.不能保证进程互斥进入临界区，会出现“饥饿”现象
- B.不能保证进程互斥进入临界区，不会出现“饥饿”现象
- C.能保证进程互斥进入临界区，会出现“饥饿”现象
- D.能保证进程互斥进入临界区，不会出现“饥饿”现象
- 注：本题为2010年全国考研题





考研题4

- 有两个并发执行的进程P1和P2，共享初值为1的变量x，P1对x加1，P2对x减1。加1和减1操作的指令序列分别如下所示。

//加1操作

//减1操作

load R1, x //取x到寄存器R1中 Load R2, x

inc R1

dec R2

store x, R1//将R1的内容存入x store x, R2

- 两个操作完成后，x的值____。 11
A、可能为-1和3 B、只能为1
C、可能为0、1或2 D、可能为-1、0、1或2





考研题5

- 某银行提供一个服务窗口和10个供顾客等待的座位。顾客到达银行时，若有空座位，则到取号机上领取一个号，等待叫号。取号机每次仅允许一个顾客使用。当营业员空闲时，通过叫号选取一位顾客，并为其服务。顾客及营业员的活动描述如下：
 - cobegin
 - { process 顾客
 - { 从取号机获取一个号码;
 - 等待叫号;
 - 获得服务; }





考研题5-2

process 营业员

```
{ while (TRUE)
  { 叫号;
    为顾客服务; } }
```

}coend

请添加必要的信号量和P、V（或wait（）、signal（））操作，实现上述过程中的互斥与同步。要求写出完整的过程，说明信号量的含义并赋初值。

11





考研题5答案

```
semaphore mutex=1; //互斥使用取号机
semaphore empty=10; //空座位的数量
semaphore full=0; //已占座位的数量
semaphore service=0; //等待叫号
cobegin
{ process 顾客i
  { P(empty); P(mutex);
    从取号机获得一个号;
    V(mutex); V(full);
    P(service); // 等待叫号  }
```





考研题5答案续

process 营业员

```
{ while(TRUE)
```

```
{ P(full);
```

```
  V(empty);
```

```
  V(service);      //叫号
```

```
  为顾客服务;
```

```
}
```

```
}
```

```
}coend
```

